

# MTGNP — Magic: The Gathering Multiplayer Network Protocol

CSNETWK • S07 • T3 AY 2025–2026 • De La Salle University Manila

This is our implementation of the Magic: The Gathering Multiplayer Network Protocol (MTGNP) v1.0, which RFC 0001 defines. It is a TCP based, message oriented, client-server system for a simplified game of Magic between two players. The server is the only authority on the game state. The clients send actions and display whatever the server tells them.

We wrote the project in Python and used only the standard library. We did not use any third-party package, so there is nothing to install and nothing to build. We frame every PDU with a 4 byte big-endian length prefix and then a UTF-8 JSON payload, exactly the way the RFC asks for it.

---

## Requirements

- Python 3.9 or newer. We developed and tested the project on Python 3.14 for Windows, and the code also runs on macOS and Linux.
- No other software is needed. There are no dependencies to install, no virtual environment to create, and no compilation step.

If `python` is not on your PATH on Windows, use the `py` launcher instead. Every command below works the same way with either one.

---

## Build and Run Instructions

There is no build step. Clone or unzip the project, open a terminal in the folder that holds `server.py`, and run the files directly.

### 1. Start the server

```
python server.py --verbose
```

The server binds to port 4444 and waits in the LOBBY state. It accepts exactly 2 clients and refuses every other connection with a `SERVER_FULL` error, the way the RFC asks for.

### 2. Start the first client

Open a second terminal in the same folder:

```
python client.py --player-id player_1 --deck decks/burn.txt --verbose
```

### 3. Start the second client

Open a third terminal:

```
python client.py --player-id player_2 --deck decks/control.txt --verbose
```

After both clients have sent `PLAYER_READY`, the server shuffles both decks, sets both life totals to 20, deals 7 cards to each player, and flips a coin for the first player. The mulligan step then begins. From there on, the players play the whole game through the prompts of the client.

The two players have to use different deck files. Both decks come from one shared fixed card set, and the RFC names permanents by their card instance ID, so a card instance can only belong to one player. All 5 sample decks in `decks/` use different cards, so any two of them work together. A deck that overlaps with the deck of the opponent is rejected with `ILLEGAL_DECK`.

## Server options

| Option                                   | Meaning                                                      |
|------------------------------------------|--------------------------------------------------------------|
| <code>--host &lt;address&gt;</code>      | The interface to bind. The default is all of them.           |
| <code>--port &lt;number&gt;</code>       | The port to listen on. The default is 4444.                  |
| <code>--verbose</code> , <code>-v</code> | Start with the verbose PDU logging turned on.                |
| <code>--pretty</code>                    | Indent the JSON of the verbose output over several lines.    |
| <code>--time-limit-ms &lt;ms&gt;</code>  | The priority timeout in milliseconds. The default is 300000. |

## Client options

| Option                                   | Meaning                                                                      |
|------------------------------------------|------------------------------------------------------------------------------|
| <code>--player-id &lt;name&gt;</code>    | The name that we claim in <code>PLAYER_READY</code> . Required.              |
| <code>--deck &lt;path&gt;</code>         | The path to a deck file, for example <code>decks/burn.txt</code> . Required. |
| <code>--host &lt;address&gt;</code>      | The server address. The default is 127.0.0.1.                                |
| <code>--port &lt;number&gt;</code>       | The server port. The default is 4444.                                        |
| <code>--verbose</code> , <code>-v</code> | Start with the verbose PDU logging turned on.                                |
| <code>--pretty</code>                    | Indent the JSON of the verbose output over several lines.                    |

Run either program with `--help` to see the same list on the console.

## Verbose Mode

Verbose mode prints every PDU that we send and every PDU that we receive, on the server side and on the client side, with a clear label for the direction and for the PDU type. We can turn it on at startup, and we can also turn it on and off while the program is running.

**To turn it on at startup**, pass `--verbose`, or `-v`, to the server and to each client:

```
python server.py --verbose
python client.py --player-id player_1 --deck decks/burn.txt --verbose
python client.py --player-id player_2 --deck decks/control.txt --verbose
```

### To turn it on and off while the program runs:

- On the **server**, type `v` and press Enter. The server answers with the new state, which is either `verbose on` or `verbose off`.
- On a **client**, type `verbose` at any prompt and press Enter.

The `--pretty` option prints the JSON body indented over several lines instead of on one line. This is easier to read when we demonstrate a large `GAME_STATE_UPDATE`, but it also makes the output much longer.

## Playing a Game

The client prints the board after every update, and it asks you what to do when it is your turn to act. Type `help` at any prompt to see this same list.

| Command                                                       | What it does                                                 |
|---------------------------------------------------------------|--------------------------------------------------------------|
| <code>pass</code>                                             | Pass priority.                                               |
| <code>cast &lt;card_id&gt; [target]</code>                    | Cast a spell. The client works out the mana payment for you. |
| <code>land &lt;card_id&gt;</code>                             | Play a land. Main Phase only, and once per turn.             |
| <code>ability &lt;permanent_id&gt; [index] [target]</code>    | Activate an ability of a permanent.                          |
| <code>concede</code>                                          | Concede the game.                                            |
| <code>keep [card_id ...] / mull</code>                        | Your mulligan decision.                                      |
| <code>discard &lt;card_id&gt; ...</code>                      | Discard down to 7 cards at Cleanup.                          |
| <code>attack [creature_id ...]</code>                         | Declare attackers. No IDs means that you do not attack.      |
| <code>block [blocker_id:attacker_id ...]</code>               | Declare blockers. No pairs means no blocks.                  |
| <code>order &lt;attacker_id&gt; &lt;blocker_id&gt; ...</code> | Assign the combat damage order.                              |
| <code>yes [target] / no</code>                                | Use an optional trigger, or decline it.                      |
| <code>state</code>                                            | Print the board again.                                       |
| <code>hand</code>                                             | List the cards in your hand.                                 |
| <code>verbose</code>                                          | Turn the PDU logging on or off.                              |
| <code>help</code>                                             | Print the command list.                                      |

A deck file lists one card per line. A line is either `<count> <card base>`, for example `4 lightning_bolt`, which becomes `lightning_bolt_001` up to `lightning_bolt_004`, or one card instance ID such as `mountain_007`. A `#` starts a comment. Any file in `decks/` is a working example.

## Running the Tests

```
python -m unittest discover -s tests
```

This runs 91 tests in about 11 seconds. We split them into two files. `tests/test_mtgnp.py` starts the real server on a real socket and talks to it with PDUs that we wrote by hand. It checks the framing, the sequence numbers, the error codes, and the whole lifecycle from LOBBY to GAME\_OVER. `tests/test_rules.py` tests the rules engine directly with no sockets at all. It covers priority, the stack, combat, the mana payment, and the card effects.

## Project Structure

Each file matches one row of the Work Distribution Matrix, so the work of each member is easy to find.

| File                                            | Responsibility                                                       |
|-------------------------------------------------|----------------------------------------------------------------------|
| <code>server.py</code> , <code>client.py</code> | The entry points.                                                    |
| <code>mtgnp/protocol.py</code>                  | Framing, all 25 PDU names, error codes, phase names.                 |
| <code>mtgnp/verbose.py</code>                   | Verbose logging, and how we turn it on and off.                      |
| <code>mtgnp/cards.py</code>                     | The fixed card catalog, the keywords, and the mana sources.          |
| <code>mtgnp/state.py</code>                     | The game state, and the per-player filtering that hides information. |
| <code>mtgnp/mana.py</code>                      | Checks a declared mana payment and taps the sources.                 |
| <code>mtgnp/effects.py</code>                   | Spell effects, activated abilities, triggers, and target legality.   |
| <code>mtgnp/engine.py</code>                    | Sequence numbers, sending, waiting for actions, state-based actions. |
| <code>mtgnp/priority.py</code>                  | Priority windows, the LIFO stack, resolution and fizzling.           |
| <code>mtgnp/combat.py</code>                    | The combat sub-state machine and the damage assignment.              |
| <code>mtgnp/turns.py</code>                     | Every phase and step, in order.                                      |
| <code>mtgnp/lifecycle.py</code>                 | LOBBY, GAME_SETUP, MULLIGAN, GAME_OVER, and the session restart.     |
| <code>mtgnp/server.py</code>                    | TCP sockets, accepting exactly 2 clients, dispatch, and PONG.        |
| <code>mtgnp/client.py</code>                    | Rendering, prompts, the heartbeat, and the deck loading.             |
| <code>data/</code>                              | The master card list, which we load while the program runs.          |
| <code>decks/</code>                             | 5 sample decks.                                                      |
| <code>tests/</code>                             | The two test suites above.                                           |

The server uses one reader thread for each client socket, and those threads only push the PDUs they receive onto a queue. One game thread owns all of the rules, so no rules code needs a lock. The reader thread answers `PING` on its own, because that PDU does not touch the game state.

## Work Distribution Matrix

| Member   | Git author                                                 |
|----------|------------------------------------------------------------|
| Member 1 | festivities <admin@festivity.moe>                          |
| Member 2 | Marco Gerard D. Mendoza <marco_gerard_mendoza@dlsu.edu.ph> |
| Member 3 | awynee <164032813+awynee@users.noreply.github.com>         |

**P** marks the primary author, who designed the component and wrote it. **S** marks a supporting member, who reviewed it, tested it, or fixed parts of it.

| Task / Feature                                            | Member 1 | Member 2 | Member 3 |
|-----------------------------------------------------------|----------|----------|----------|
| TCP Server: connection handling, framing, dispatch        | <b>P</b> | S        |          |
| Game lifecycle: LOBBY, GAME_SETUP, MULLIGAN logic         | S        | <b>P</b> |          |
| Turn & phase engine (all phases/steps, transitions)       |          | S        | <b>P</b> |
| Priority & Stack logic, spell/ability resolution          | S        |          | <b>P</b> |
| Combat system (attackers, blockers, damage)               |          | S        | <b>P</b> |
| Client implementation & state rendering                   | S        |          | <b>P</b> |
| PDU serialization and deserialization (all 25 PDU types)  | <b>P</b> | S        |          |
| Error handling, PING/PONG heartbeat, disconnect logic     | <b>P</b> |          | S        |
| Verbose mode (client + server PDU logging, toggle on/off) | <b>P</b> |          | S        |
| Card catalog, mana payment & card effects                 |          | <b>P</b> | S        |
| Game state management & hidden-information filtering      |          | <b>P</b> | S        |
| Testing & interoperability                                | <b>P</b> | S        | S        |
| README / documentation / AI disclosure                    | S        | <b>P</b> | S        |

We did not attempt the two bonus criteria, which are implementing every card ability and adding extra features such as a graphical interface or a spectator client. This submission aims for the 100 point base rubric.

## AI Usage

We used AI tools while we worked on this machine problem. The table below lists them and explains how we used each one.

| Tool                                                          | How we used it                                                                                                                                                          |
|---------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Claude (Anthropic), through the Claude Code command line tool | Drafting and reviewing large parts of the Python source, explaining the sections of the RFC that contradict themselves, generating test cases, and writing this README. |

| Tool                                           | How we used it                                                                                                                          |
|------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| OpenCode, an open-source terminal coding agent | We used this in an earlier attempt at the same project, mostly for the module layout and for the first version of the PDU framing code. |

How we worked with these tools:

- We wrote our prompts around the RFC itself. Where the RFC was unclear, we chose the reading first and then asked the tool to implement that reading, instead of letting the tool decide for us.
- We read every piece of generated code line by line, edited it to match the rest of the project, and tested it. The 91 tests in `tests/` are the main way we checked the output, and we also played full games over real sockets.
- The deviations in the next section are our own decisions. We traced each one back to a specific section of the RFC before we accepted it.
- We did not share any AI output with another group and did not take any from one, and we did not reuse a session from another student.

Every member can explain any part of the submission, including the parts that we drafted with the help of AI.

## Known Limitations and Deviations from the RFC

### Where the RFC contradicts itself

Section 5.3 says that Section 10.2 is the authority for field names, so we follow Section 10.2 wherever the prose examples do not agree with it. Our client accepts both spellings, so that it can still talk to the server of another group.

| Field             | What we send, per §10.2                 | What the prose examples show    |
|-------------------|-----------------------------------------|---------------------------------|
| Hand              | <code>{"player_1": [...]}</code> object | A bare array                    |
| Land flag         | <code>land_played_this_turn</code>      | <code>land_played</code>        |
| Creature flag     | <code>summoning_sick</code>             | <code>summoning_sickness</code> |
| State change kind | <code>change_type</code>                | <code>type</code>               |

### Deliberate protocol decisions

- **The sequence number is a plain counter** that goes up by 1 for every PDU the server sends, and it keeps counting across games in the same session. Section 5.4 allows this. A broadcast to both players uses one sequence number, while each `GAME_STATE_UPDATE` that we filter for one player gets its own.
- **After an invalid action, the new `PRIORITY_GRANT` carries a new sequence number**, and that new number becomes the valid token. This matches the worked example in Section 5.4. The words in Section 11 say that the same number is used again, which does not agree with the example, so we followed the example.
- **The declaration steps have no request PDU.** `DECLARE_ATTACKERS`, `DECLARE_BLOCKERS` and `ASSIGN_DAMAGE_ORDER` echo the sequence number of the `PHASE_TRANSITION`, the way Section 5.4 shows it. No `PRIORITY_GRANT` comes before them.

- **We send a state update after every accepted action.** The RFC does not ask for this, but `STACK_PUSH` says nothing about zones, so without that extra update a client cannot tell that a card left a hand.
- **The visible state carries one extra key, `combat`,** which holds the attackers, the blocks, and the damage order. Section 10.2 does not name it, but all of it is public information, and without it the attacking player has no way to learn how we blocked them, which they need before they can send `ASSIGN_DAMAGE_ORDER`. We renamed nothing, so a stricter client can just ignore the key.
- **The two deck lists must not overlap,** for the reason we gave earlier. The deck check also rejects a card whose copy count is higher than the "Copies in Set" column of the master card list. Both cases return `ILLEGAL_DECK`.
- **Summoning sickness goes away at the Untap Step of the controller,** which follows Section 3, and not at Cleanup as one of the examples shows.
- **The priority timeout is 300000 ms by default** instead of the 60000 ms in the examples, so that a live demo does not time out while a player is thinking. The `--time-limit-ms` option changes it.
- **A disconnect or a priority timeout ends the game** with `GAME_OVER(DISCONNECT)`. The player who is still there keeps their connection, the server goes back to LOBBY, and the empty slot opens for another client. This is what Section 4.2 asks for after a timeout. Joining a game that is already running again is not supported.
- **The client copies the phase, the turn number and the active player in from `PHASE_TRANSITION`,** because the server does not send a state update after every transition.

## Card effects

The rubric asks for at least 5 card effects. We implemented 16 spell effects, 4 activated abilities, which are Prodigal Sorcerer, Royal Assassin, Rod of Ruin and Millstone, and 3 triggered abilities, which are Gray Merchant, Gravedigger and Goblin Guide. We handle the mana abilities in the background, which Section 7.5 allows, and those are the basic lands, Llanowar Elves, Elvish Mystic and Sol Ring. The keywords that we enforce are haste, first strike, double strike, defender, flying and vigilance.

We did **not** implement the following. When a player casts one of these, we reject it with `ILLEGAL_ACTION` instead of quietly doing nothing:

- Keywords and mechanics: prowess, protection, hexproof, kicker, madness, suspend, regenerate, illusion, and Auras such as Pacifism. Section 9.7 of the RFC itself leaves trample out.
- Spells: Skullcrack, Rift Bolt, Mana Leak, Vines of Vastwood, Naturalize, Dark Ritual, Ponder, Rampant Growth, Mind Rot, and the land search half of Path to Exile. Healing Salve uses its first mode only.

Most of these share the same reason. They need the player to make a choice in the middle of resolving a spell, and the RFC defines no PDU that asks that question. We stayed away from the ones that would have made us invent a message that the protocol does not have. Implementing all of them is bonus work, and this submission does not attempt it.

---

## Rebuilding This Document

We build `README.pdf` from `README.md`, so the Markdown file is the one that matters. After you edit it, run:

```
build_readme.bat
```

The script turns `README.md` into HTML with `tools/md2pdf.py`, which uses only the Python standard library. It then prints that HTML to `README.pdf` with a headless Microsoft Edge or Google Chrome, whichever one it finds first. On macOS or Linux, or if you would rather not use the batch file, run the same step directly:

```
python tools/md2pdf.py README.md README.pdf
```

Neither script is part of the protocol implementation. They only exist to keep the PDF that we submit in step with the Markdown source.